

TP 1 – Construction d’une API via Spring Boot (durée 4H)

Le but de ce TP est de construire à l’aide du Framework Spring Boot une application de gestion de personnel d’une entreprise. Cette API sera construite avec le Framework Spring Boot. Elle sera sollicitée via une interface HTML et un programme JavaScript contenant des fonctions AJAX qui vont solliciter ces web services. Le premier objectif est de créer l’API et de la tester. Ensuite, nous créerons les fichiers HTML et JS nécessaires au fonctionnement de l’application.

I- Présentation des web services à développer

Voici ci-après les différents web services à développer avec leurs routes et leurs fonctionnalités. *Les formats des données transmises dans les requêtes et dans les réponses seront au format JSON.* Une entreprise dispose de plusieurs départements qui contiennent différents collaborateurs. L’application permettra de gérer les collaborateurs au sein de ces départements.

Ressource Departements

Méthode	Url	Description
GET	/api/departements	retourne la liste des departements de l’entreprise au format JSON.
GET	/api/departements/ [ID]	retourne le département d’identifiant id.
POST	/api/departements/create	enregistre un département en base de données
DELETE	/api/departements/ [ID]	supprime le département d’identifiant id.
PUT	/api/collaborateurs/ [ID]	modifie le département d’identifiant id.

Ressource Collaborateurs

Les formats des données transmises dans les requêtes et dans les réponses seront au format JSON.

Méthode	Url	Description
GET	/api/collaborateurs	retourne la liste des collaborateurs
GET	/api/collaborateurs/ [ID_DEPARTEMENT]	retourne la liste des collaborateurs dont le departement a l’identifiant <i>ID_DEPARTEMENT</i>
GET	/api/collaborateurs/[MATRICULE]	retourne les informations d’un collaborateur possédant un certain matricule.
GET	/api/collaborateurs/nom/[NOM]	retourne la liste des collaborateurs au format JSON dont le nom est <i>NOM</i>
PUT	/api/collaborateurs/ [MATRICULE]	modifie un collaborateur dont le matricule est MATRICULE.
POST	/api/collaborateurs/create	enregistre un collaborateur
DELETE	/api/collaborateurs/delete/ [MATRICULE]	supprimer un collaborateur

I- Génération du projet via Spring Initializr

Rendez-vous sur le site <http://start.spring.io>.

Générer un projet avec les informations suivantes :

- Choisir un projet Maven
- Version de Spring Boot : 3.14
- Group : com.iut
- Artifact : collaborateurs
- Dependencies : Spring Web, JPA SQL, H2 Database, Spring Boot Devtools.

Ensuite, cliquez sur « Generate » et décompressez l'archive dans le workspace.

The screenshot shows the Spring Initializr web interface. On the left, under 'Project', 'Maven' is selected. Under 'Language', 'Java' is selected. Under 'Spring Boot', version '3.1.4' is selected. The 'Project Metadata' section contains: Group (com.iut), Artifact (collaborateurs), Name (collaborateurs), Description (Demo project for Spring Boot), and Package name (com.iut.collaborateurs). Under 'Packaging', 'Jar' is selected. At the bottom, 'Java' is selected with version '17'. On the right, the 'Dependencies' section lists: 'Spring Web' (WEB), 'Spring Data JPA' (SQL), 'H2 Database' (SQL), and 'Spring Boot DevTools' (DEVELOPER TOOLS). Each dependency has a brief description. A button 'ADD DEPENDENCIES... CTRL + B' is at the top right of the dependencies section.

Puis, importez le projet via Eclipse : File -> import -> existing Maven Projects, allez chercher votre projet dans votre workspace et finish.

II- Organisation des fichiers dans le projet

Voici, ci-après l'organisation finale des fichiers dans notre projet. Vous pourrez vous y référer dans la suite. Cependant, vous pouvez déjà créer les packages suivants :

- entity
- listener
- repository
- service

- ▼  src/main/java
 - ▼  com.iut.collaborateurs
 - >  CollaborateursApplication.java
 - ▼  com.iut.collaborateurs.controllers
 - >  CollaborateurController.java
 - >  DepartementsController.java
 - ▼  com.iut.collaborateurs.entity
 - >  Collaborateur.java
 - >  Departement.java
 - ▼  com.iut.collaborateurs.listener
 - >  DataInitializerListener.java
 - ▼  com.iut.collaborateurs.repository
 - >  CollaborateurRepository.java
 - >  DepartementRepository.java
 - ▼  com.iut.collaborateurs.service
 - >  CollaborateurService.java
 - >  DataInitializerService.java
 - >  DepartementService.java

III- Mise en place des entités

Nous partirons de façon simplifiée avec deux entités fondamentales :

- Les collaborateurs
- Les départements

Créez les classe « Collaborateur » et « Departement » dans le package « entity ».

Les collaborateurs sont caractérisés par :

- Un identifiant -un entier
- Un matricule – une chaîne de caractères
- Un nom – une chaîne de caractères
- Un prénom – une chaîne de caractères
- Une adresse postale – une chaîne de caractères
- Un email professionnel – une chaîne de caractères
- Un numéro de sécurité sociale (une chaîne de caractères)
- Une photo (url de la photo) – une chaîne de caractères
- Un attribut permettant de savoir s’il est actif ou non – un booléen
- L’intitulé du poste – une chaîne de caractères
- L’identifiant du département dans lequel il est affilié - un entier
- La civilité – une chaîne de caractères
- Le nom de la banque – une chaîne de caractères
- Le BIC du compte bancaire – une chaîne de caractères
- Le BAN du compte bancaire – une chaîne de caractères

Les départements son caractérisé par :

- Un identifiant – un entier
- Un nom – une chaîne de caractères

Dans un package appelé « entity », créer les deux classes « Département » et « Collaborateur ». Ces classes contiendront les getters et setters correspondant à tous les attributs, ainsi que des constructeurs avec au moins un sans paramètre.

IV- Mise en place d'une base de données H2

Nous partirons sur une base H2. Ce type de base est qualifié « in memory », elle a donc des capacités de stockage limitées, mais est suffisant pour le but que nous nous fixons d'atteindre. Les syntaxes des requêtes sont très proches de SQL. .

1- Configuration du fichier application.properties

Lors de l'initialisation de votre projet via Spring Initializr, un fichier « application.properties » a été généré. Ce fichier va nous permettre de préciser quelques paramétrages utiles au bon fonctionnement de la connexion à notre base de données H2. Vous le trouverez dans le dossier « src/main/resources ».

Voici les informations à renseigner dans ce fichier :

```
# datasource
# url de la base de données H2 "testdb"
spring.datasource.url = jdbc:h2:mem:testdb
# driver utilisé
spring.datasource.driver-class-name = org.h2.Driver
# login d'accès à la base de données
spring.datasource.username = sa
# mot de passe d'accès à la base de données
spring.datasource.password =
# génération de la base de données sur la base des entités définies
spring.jpa.generate-ddl=true
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
# outils de logging
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

2- Création des tables via les classes

En utilisant les annotations JPA adaptées, annotez les deux classes « Collaborateur » et « Département » de telle façon qu'elles génèrent une deux tables « collaborateurs » et « departements » dans la base de données H2. Pour cela :

La classe sera annotée avec :

- **@Entity** une instance d'une classe qui sera persistante (que l'on pourra sauvegarder dans / charger depuis une base de données relationnelle) ...
- **@Table** : permet de générer la table correspondant à la classe. Vous pouvez préciser en paramètre de cette annotation le nom de la table qui correspondra à la classe. Par exemple :
@Entity
@Table (name="departement")
public class Departement {...}

Les attributs seront annotés avec :

@Column. Vous pouvez préciser en paramètre de @Column le nom que vous souhaitez donner à l'attribut de la table. Par exemple :

```
@Column (name="nom_departement")  
private String nomDepartement;
```

Pour la déclaration de la clé primaire auto incrémentée, faire précéder la déclaration de l'attribut par l'annotation @Id puis par l'annotation @GeneratedValue qui précisera la stratégie de génération appropriée. Par exemple :

```
@Id  
@GeneratedValue (strategy = GenerationType.AUTO)  
@Column (name="id_departement")  
private int idDepartement;
```

Suivez les mêmes règles pour annoter la classe Collaborateur.

V- Création du repository

Dans un package nommé repository, créer la classe DepartementRepository. Annotez-la avec @Repository et @Transactional (assure que les accès à la base de données seront transactionnelles).

Nous rappelons que @Repository n'est qu'une spécialisation sémantique de @Component qui permet de charger les bean Spring afin qu'ils soient utilisables dans le contexte de l'application.

Pour pouvoir bénéficier des avantages d'un repository JPA, la classes DepartementRepository devra étendre : JpaRepository<Departement, Integer>.

Nous héritons ainsi de toutes les méthodes implémentées par défaut dans la classe JpaRepository. Voici la documentation en ligne : <https://docs.spring.io/spring-data/jpa/docs/current/api/org/springframework/data/jpa/repository/JpaRepository.html>

Vous remarquerez en particulier que JpaRepository hérite CrudRepository ...

Vous vous retrouverez alors avec une classe très simple !

```
@Repository
@Transactional
public interface DepartementRepository extends JpaRepository<Departement,
Integer> {

}
```

VI- Création d'un service

Dans le package service, créer la classe DepartementService. Vous l'annoterez avec @Service.

@Service est une spécialisation de l'annotation @Component.

Les services représentent la couche métier de notre application. Ici, il n'y a rien de particulier au niveau métier, et cette classe ne sera finalement qu'une encapsulation du repository.

Nous aurons donc besoin d'un objet de type DepartementRepository. C'est ici qu'intervient le concept fondamental de Spring : l'injection de dépendance. Le cycle de vie des composants Spring (qui sont des bean Spring) nous permet ici d'utiliser un objet de type DepartementRepository dans notre classe DepartementService (c'est là que se trouve la dépendance).

Pour faire appel à un tel composant, nous utiliserons l'annotation **@Autowired** au niveau de l'attribut de type DepartementRepository :

```
@Service
public class DepartementService {

    @Autowired
    private DepartementRepository departementRepository;

    ...
}
```

En utilisant l'objet « departementRepository », écrivez les méthodes suivantes :

- **public** List<Departement> getAllDepartements () // récupère tous les départements présents en base de données.
- **public** Departement getDepartementById (int id)// récupère le département possédant un id donné en paramètre.
- **public** Departement saveDepartement (Departement departement)// permet d'enregistrer un département donné en paramètre dans la base de données.

VII- Création d'un controller

Dans le package « controller », créer la classe « DepartementController ».

Cette classe sera annotée par **@RestController**. Puis, nous ajoutons à la suite de cette annotation, toujours au niveau de la classe : **@RequestMapping ("/api/departements")**.

Nous rappelons que l'adresse de base du server précédera donc `"/api/departements"` et que tous les services développés dans ce contrôleur auront leur uri aussi précédées de `"/api/departements"`.

Nous pouvons maintenant aborder le développement des services à proprement parler.

Les différents web services correspondront à différentes méthodes de cette classe. Lors de la sollicitation d'une url spécifique, le code se branchera sur la méthode associée à cette url et le code en sera exécuté.

Selon la structure en couches d'une application Spring, nous aurons besoin dans ce controller du service relatif aux départements. Il s'agit donc d'une dépendance. Celle-ci sera injectée encore une fois via **@Autowired**.

```
@RestController
@RequestMapping ("/api/departements")
public class DepartementsController {

    @Autowired
    private DepartementService departementService;

    @GetMapping ("/{id}")
    public Departement listerDepartementParId(@PathVariable("id") String id) {
        return this.departementService.getDepartementById(Integer.valueOf(id));
    }
    ...}
```

La méthode *listerDepartementParId* retourne le département qui a pour identifiant celui qui est donné en paramètre. Le corps de la méthode fait appel au service qui permet de retrouver un département d'id donné.

L'annotation **@GetMapping ("/ {id}")**, est relative à la méthode HTTP « GET ». Elle permet de préciser le complément d'url qui permettra de solliciter le service permettant de retrouver un département donné. Ici, il s'agit de « id » mis entre accolades, car pouvant prendre en tant que paramètre d'url différentes valeurs selon le besoin.

La récupération de la valeur de l'id dans la méthode se fait via le paramètre de la méthode précédé de **@PathVariable** ("nom_du_parametre").

Selon la méthode http définie pour le web service donné, nous pourrons faire appel à **@PostMapping**, **@PutMapping**, etc. Dans ces deux derniers cas, il nous faudrait récupérer éventuellement le corps de la requête HTTP, chose que l'on ferait avec **@RequestBody**.

Vous pourrez de même écrire la méthode suivante, sachant qu'elle est sollicitée sans complément d'url :

```
@GetMapping
public List<Departement> listerDepartements ()
```

VIII- Peuplement initial de la base de données

Nous allons créer un service spécifique pour l'initialisation de notre base de données H2.

Créer dans le package « service », un service appelé : `DataInitializerService`. Ce service initialisera la table « departement » et la table « collaborateurs ». Nous prendrons ici l'exemple de la table « departement ».

Déclarer en **@Autowired** un objet de type `DepartementService`. Créer une méthode void `initialiser ()`. Vous créerez dans cette méthode un certain nombre de départements (au moins 5). Vous utiliserez le bean déclaré précédemment pour enregistrer ces départements.

Pour initialiser le peuplement de la base de données au lancement de l'application, vous créerez dans le package « listener » la classe `DataInitializerListener`, que nous donnons ici. Elle sera automatiquement appelée au lancement de l'application.

```
@Component
public class DataInitializerListener {

    @Autowired
    private DataInitializerService initialiserDataServiceDev;

    @EventListener(ContextRefreshedEvent.class)
    public void contextRefreshedEvent() {
        initialiserDataServiceDev.initialiser();
    }
}
```

Vous pourrez créer le repository pour les collaborateurs en suivant les mêmes règles. Composez les noms de fonction qui permettent l'accès aux données selon les indications de routes spécifiées plus en IV dans le listing de web services.

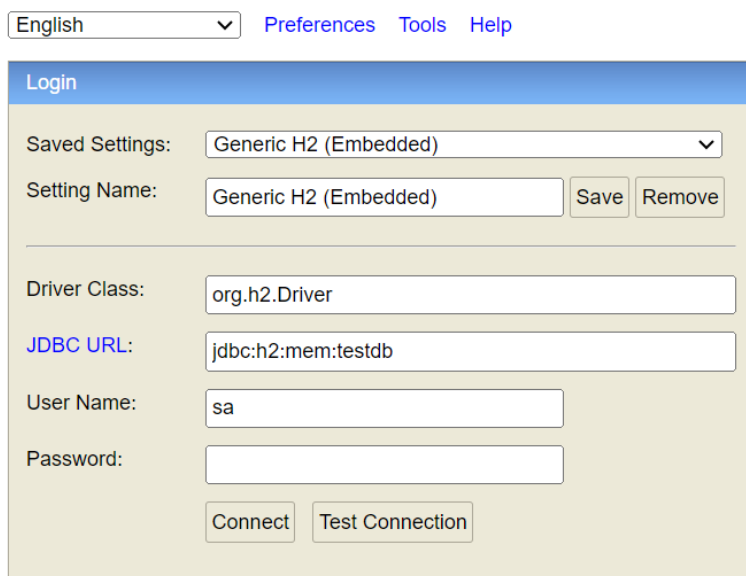
IX- Visualisation des données insérées

Nous pouvons visualiser les données qui ont été intégrées à la base de données en utilisant la console H2.

Pour cela, configurer dans le dossier « resources » le fichier « application.properties ». Rajouter ces lignes :

spring.h2.console.enabled=true
spring.h2.console.path=/h2

La console H2 sera disponible via votre navigateur à l'adresse : <http://localhost:8080/h2>
Renseignez l'adresse de votre base de données.



English Preferences Tools Help

Login

Saved Settings: Generic H2 (Embedded) ▼

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:mem:testdb

User Name: sa

Password:

Connect Test Connection

X- Test des web services

Plusieurs outils permettent de tester des web services. L'un des plus connu est un outil de Google qui l'appelle « Postman ». C'est une application qui s'installe sur votre ordinateur.

D'autres outils en ligne, comme : <https://www.site24x7.com/tools/restapi-tester.html> ou autres!

Prenez l'habitude de tester tous vos web services dans ce genre d'outils !

XI- Gestion des collaborateurs

Suivez la démarche qui a été proposée pour les départements pour mettre en place les différents services relatifs aux collaborateurs, en effectuant les recherches nécessaires au besoin.

XII- Sollicitation de l'API via une interface

1- Cors

Afin de gérer la sécurité en ce qui concerne les origines croisées de requêtes, Spring propose un bean à intégrer dans la classe de lancement de l'application (celle contenant la méthode « run »).

@Bean

```
public WebMvcConfigurer corsConfigurer() {  
    return new WebMvcConfigurer() {  
        @Override  
        public void addCorsMappings(CorsRegistry registry) {  
            registry.addMapping("/api/**")  
                .allowedHeaders("Origin", "Content-Type", "Accept")  
                .allowedOrigins("*")  
                .allowedMethods("*");  
        }  
    };  
}
```

2- Instructions pour la mise en place de l'interface utilisateur

- Au lancement de l'application, la liste des collaborateurs est affichée.
- La liste des départements est accessible via une liste déroulante. Le choix d'une option permet de filtrer les personnes de ce département dans le tableau des collaborateurs.
- Au click sur le bouton de modification, attaché à un collaborateur dans le tableau, un formulaire pré-rempli apparaît. Le bouton sauvegarder permet d'enregistrer les modifications en base de données.
- Au click sur un bouton, il est possible d'ajouter un collaborateur, un formulaire vide apparaît alors
- Il est possible de filtrer la liste des collaborateurs par nom.

Lister les collaborateurs :

Gestion des collaborateurs

Rechercher

Filtrer par département

Tous

Rechercher par nom

Rechercher par nom

Rechercher

Détails des Collaborateurs

Photo	Matricule	Nom	Prénom	Modifier	Supprimer
	M0	César	Jules		
	M1	Bono	Jean		
	M2	Smith	Lucie		
	M3	Sim	Sophie		

Ajouter un collaborateur

Modification d'un collaborateur :

Gestion des collaborateurs

Rechercher

Filtrer par département

Tous

Rechercher par nom

Rechercher par nom

Rechercher

Détails des Collaborateurs

Photo	Matricule	Nom	Prénom	Modifier	Supprimer
	M0	César	Jules		
	M1	Bono	Jean		
	M2	Smith	Lucie		
	M3	Sim	Sophie		

Ajouter un collaborateur

Modifier le collaborateur M0

Nom

César

Prénom

Jules

Civilité

M

Département

Ressources Humaines

Intitulé du poste

Directeur

Adresse mail professionnelle

Jules.César@societe.com

Adresse

4 rue Edith Piaf, 44800 Saint-Herblain

Numéro de sécurité sociale

123456789123456

Adresse de la photo

https://bootply.com/img/Content/User_1.jpg

Banque

Caisses d'Epargne

BIC

BICS233504412

BAN

BAN125436544

Sauvegarder

Enregistrement d'un collaborateur :

Gestion des collaborateurs

Filtrer par département

Tous

Rechercher

Rechercher par nom

Rechercher

Détails des Collaborateurs

Photo	Matricule	Nom	Prénom	Modifier	Supprimer
	M0	César	Jules		
	M1	Bono	Jean		
	M2	Smith	Lucie		
	M3	Sim	Sophie		

Ajouter un collaborateur

Ajouter un collaborateur

Nom

Prénom

Civilité

Département

Intitulé du poste

Adresse mail professionnelle

Adresse

Numéro de sécurité sociale

Adresse de la photo

Banque

BIC

BAN

Sauvegarder